

TASK-5 : EXPLORATORY DATA ANALYSIS (EDA)

I have used TITANIC Datasets (train.csv and test.csv) from Kaggle.com

1) Import The Necessary Library :

```
In [39]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

# Ignore all warnings
warnings.filterwarnings("ignore")
```

2) Load the dataset from a CSV file

```
In [40]: train = pd.read_csv('train.csv')
test = pd.read_csv('test.csv')
print("Datasets loaded successfully.")
```

Datasets loaded successfully.

3) Display the first 5 rows and a summary of the data

```
In [42]: print("\nInitial train.csv Dataset Preview:")
print(train.head())
print("\nInformation summary of train.csv:")
train.info()
train
```

Initial train.csv Dataset Preview:

	PassengerId	Survived	Pclass	\
0	1	0	3	
1	2	1	1	
2	3	1	3	
3	4	1	1	
4	5	0	3	

	Name	Sex	Age	SibSp	\
0	Braund, Mr. Owen Harris	male	22.0	1	
1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	
2	Heikkinen, Miss. Laina	female	26.0	0	
3	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	
4	Allen, Mr. William Henry	male	35.0	0	

	Parch	Ticket	Fare	Cabin	Embarked
0	0	A/5 21171	7.2500	NaN	S
1	0	PC 17599	71.2833	C85	C
2	0	STON/O2. 3101282	7.9250	NaN	S
3	0	113803	53.1000	C123	S
4	0	373450	8.0500	NaN	S

Information summary of train.csv:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 891 entries, 0 to 890

Data columns (total 12 columns):

#	Column	Non-Null Count	Dtype
0	PassengerId	891 non-null	int64
1	Survived	891 non-null	int64
2	Pclass	891 non-null	int64
3	Name	891 non-null	object
4	Sex	891 non-null	object
5	Age	714 non-null	float64
6	SibSp	891 non-null	int64
7	Parch	891 non-null	int64
8	Ticket	891 non-null	object
9	Fare	891 non-null	float64
10	Cabin	204 non-null	object
11	Embarked	889 non-null	object

dtypes: float64(2), int64(5), object(5)

memory usage: 83.7+ KB

Out[42]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599 7
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803 5
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450
...	...	...	...	...	...	...	...	...	...
886	887	0	2	Montvila, Rev. Juozas	male	27.0	0	0	211536 1
887	888	1	1	Graham, Miss. Margaret Edith	female	19.0	0	0	112053 3
888	889	0	3	Johnston, Miss. Catherine Helen "Carrie"	female	NaN	1	2	W./C. 6607 2
889	890	1	1	Behr, Mr. Karl Howell	male	26.0	0	0	111369 3
890	891	0	3	Dooley, Mr. Patrick	male	32.0	0	0	370376

891 rows × 12 columns



In [43]:

```
print("\nInitial test.csv Dataset Preview:")
print(test.head())
print("\nInformation summary of test.csv:")
```

```
test.info()
test
```

Initial test.csv Dataset Preview:

	PassengerId	Pclass	Name	Sex	\
0	892	3	Kelly, Mr. James	male	
1	893	3	Wilkes, Mrs. James (Ellen Needs)	female	
2	894	2	Myles, Mr. Thomas Francis	male	
3	895	3	Wirz, Mr. Albert	male	
4	896	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	

	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	34.5	0	0	330911	7.8292	NaN	Q
1	47.0	1	0	363272	7.0000	NaN	S
2	62.0	0	0	240276	9.6875	NaN	Q
3	27.0	0	0	315154	8.6625	NaN	S
4	22.0	1	1	3101298	12.2875	NaN	S

Information summary of test.csv:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 418 entries, 0 to 417

Data columns (total 11 columns):

#	Column	Non-Null Count	Dtype
0	PassengerId	418 non-null	int64
1	Pclass	418 non-null	int64
2	Name	418 non-null	object
3	Sex	418 non-null	object
4	Age	332 non-null	float64
5	SibSp	418 non-null	int64
6	Parch	418 non-null	int64
7	Ticket	418 non-null	object
8	Fare	417 non-null	float64
9	Cabin	91 non-null	object
10	Embarked	418 non-null	object

dtypes: float64(2), int64(4), object(5)

memory usage: 36.1+ KB

Out[43]:

	PassengerId	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare
0	892	3	Kelly, Mr. James	male	34.5	0	0	330911	7.8292
1	893	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	363272	7.0000
2	894	2	Myles, Mr. Thomas Francis	male	62.0	0	0	240276	9.6875
3	895	3	Wirz, Mr. Albert	male	27.0	0	0	315154	8.6625
4	896	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1	3101298	12.2875
...	...	...	...	...	...	...	...	...	...
413	1305	3	Spector, Mr. Woolf	male	NaN	0	0	A.5. 3236	8.0500
414	1306	1	Oliva y Ocana, Dona. Fermina	female	39.0	0	0	PC 17758	108.9000
415	1307	3	Saether, Mr. Simon Sivertsen	male	38.5	0	0	SOTON/O.Q. 3101262	7.2500
416	1308	3	Ware, Mr. Frederick	male	NaN	0	0	359309	8.0500
417	1309	3	Peter, Master. Michael J	male	NaN	1	1	2668	22.3583

418 rows × 11 columns



In [44]: `print(train.shape)`
`print(test.shape)`

(891, 12)

(418, 11)

4) Identify or Checking missing values

In [45]: `# Checking for missing values for train.csv`

```
train.isnull().sum() # isna() --- to find missing values or df.isna().sum()
```

```
Out[45]: PassengerId      0
         Survived        0
         Pclass          0
         Name            0
         Sex             0
         Age            177
         SibSp           0
         Parch           0
         Ticket          0
         Fare            0
         Cabin          687
         Embarked        2
         dtype: int64
```

The 'Cabin' column contains numerous null values, so we will drop it.

```
In [47]: # Checking for missing values for train.csv
```

```
test.isnull().sum() # isna() --- to find missing values or df.isna().sum()
```

```
Out[47]: PassengerId      0
         Pclass          0
         Name            0
         Sex             0
         Age             86
         SibSp           0
         Parch           0
         Ticket          0
         Fare            1
         Cabin          327
         Embarked        0
         dtype: int64
```

The 'Cabin' column contains numerous null values, so we will drop it.

5) Handling Missing Values

```
In [48]: #For columns with very few missing values, you can drop the entire row
train.drop(columns=['Cabin'],inplace=True)
test.drop(columns=['Cabin'],inplace=True)

# Again Checking for missing values

train.isnull().sum()
```

```
Out[48]: PassengerId      0
         Survived        0
         Pclass          0
         Name            0
         Sex             0
         Age            177
         SibSp           0
         Parch           0
         Ticket          0
         Fare            0
         Embarked        2
         dtype: int64
```

```
In [49]: # Again Checking for missing values
```

```
test.isnull().sum()
```

```
Out[49]: PassengerId      0
         Pclass          0
         Name            0
         Sex             0
         Age            86
         SibSp           0
         Parch           0
         Ticket          0
         Fare            1
         Embarked        0
         dtype: int64
```

6) Filling missing values (Imputation)

```
In [52]: # Filling missing values
train['Embarked'].fillna('S', inplace=True)
```

```
# Again Checking for missing values
```

```
train.isnull().sum()
```

```
Out[52]: PassengerId      0
         Survived        0
         Pclass          0
         Name            0
         Sex             0
         Age            177
         SibSp           0
         Parch           0
         Ticket          0
         Fare            0
         Embarked        0
         dtype: int64
```

```
In [53]: # Filling missing values
test['Fare'].fillna(test['Fare'].mean(), inplace=True)
```

```
# Again Checking for missing values
```

```
test.isnull().sum()
```

```
Out[53]: PassengerId    0
         Pclass        0
         Name          0
         Sex           0
         Age          86
         SibSp         0
         Parch         0
         Ticket        0
         Fare          0
         Embarked      0
         dtype: int64
```

7) Creating a DataFrame for Easier Manipulation

```
In [55]: df=pd.concat([train,test],sort=True).reset_index(drop=True)
```

```
In [56]: print(df.shape)
```

```
(1309, 11)
```

```
In [58]: df.head()
```

```
Out[58]:
```

	Age	Embarked	Fare	Name	Parch	PassengerId	Pclass	Sex	SibSp	Surv
0	22.0	S	7.2500	Braund, Mr. Owen Harris	0	1	3	male	1	
1	38.0	C	71.2833	Cumings, Mrs. John Bradley (Florence Briggs Th...	0	2	1	female	1	
2	26.0	S	7.9250	Heikkinen, Miss. Laina	0	3	3	female	0	
3	35.0	S	53.1000	Futrelle, Mrs. Jacques Heath (Lily May Peel)	0	4	1	female	1	
4	35.0	S	8.0500	Allen, Mr. William Henry	0	5	3	male	0	

0	22.0	S	7.2500	Braund, Mr. Owen Harris	0	1	3	male	1	
1	38.0	C	71.2833	Cumings, Mrs. John Bradley (Florence Briggs Th...	0	2	1	female	1	
2	26.0	S	7.9250	Heikkinen, Miss. Laina	0	3	3	female	0	
3	35.0	S	53.1000	Futrelle, Mrs. Jacques Heath (Lily May Peel)	0	4	1	female	1	
4	35.0	S	8.0500	Allen, Mr. William Henry	0	5	3	male	0	

The 'Age' column contains numerous null values, we need to fill in the null values in the 'Age' column with meaningful values.

```
In [65]: df.Age.fillna(df.Age.mean(),inplace=True)

# Again Checking for missing values
```



```
df.isnull().sum()
```

```
Out[65]: Age          0
Embarked      0
Fare          0
Name          0
Parch         0
PassengerId   0
Pclass        0
Sex           0
SibSp         0
Survived      418
Ticket        0
dtype: int64
```

```
In [66]: df.head()
```

```
Out[66]:
```

	Age	Embarked	Fare	Name	Parch	PassengerId	Pclass	Sex	SibSp	Surv
0	22.0	S	7.2500	Braund, Mr. Owen Harris	0	1	3	male	1	
1	38.0	C	71.2833	Cumings, Mrs. John Bradley (Florence Briggs Th...	0	2	1	female	1	
2	26.0	S	7.9250	Heikkinen, Miss. Laina	0	3	3	female	0	
3	35.0	S	53.1000	Futrelle, Mrs. Jacques Heath (Lily May Peel)	0	4	1	female	1	
4	35.0	S	8.0500	Allen, Mr. William Henry	0	5	3	male	0	

The 'Survived' column contains numerous null values, we need to fill in the null values in the 'Survived' column with mode.

```
In [68]: # Impute with the mode

mode_survived = df['Survived'].mode()[0]
df['Survived'].fillna(mode_survived, inplace=True)

# Again Checking for missing values
df.isnull().sum()
```

Out[68]: Age 0
Embarked 0
Fare 0
Name 0
Parch 0
PassengerId 0
Pclass 0
Sex 0
SibSp 0
Survived 0
Ticket 0
dtype: int64

```
In [69]: df.head()
```

Out[69]:

	Age	Embarked	Fare	Name	Parch	PassengerId	Pclass	Sex	SibSp	Surv
0	22.0	S	7.2500	Braund, Mr. Owen Harris	0	1	3	male	1	
1	38.0	C	71.2833	Cumings, Mrs. John Bradley (Florence Briggs Th...	0	2	1	female	1	
2	26.0	S	7.9250	Heikkinen, Miss. Laina	0	3	3	female	0	
3	35.0	S	53.1000	Futrelle, Mrs. Jacques Heath (Lily May Peel)	0	4	1	female	1	
4	35.0	S	8.0500	Allen, Mr. William Henry	0	5	3	male	0	

```
In [71]: df.tail()
```

Out[71]:

	Age	Embarked	Fare	Name	Parch	PassengerId	Pclass	Sex	Sib
1304	29.881138	S	8.0500	Spector, Mr. Woolf	0	1305	3	male	
1305	39.000000	C	108.9000	Oliva y Ocana, Dona. Fermina	0	1306	1	female	
1306	38.500000	S	7.2500	Saether, Mr. Simon Sivertsen	0	1307	3	male	
1307	29.881138	S	8.0500	Ware, Mr. Frederick	0	1308	3	male	
1308	29.881138	C	22.3583	Peter, Master. Michael J	1	1309	3	male	

8) Check and fix data types

In [70]: *# Check the data types*

```
print(df.dtypes)
```

```
Age          float64
Embarked     object
Fare         float64
Name         object
Parch        int64
PassengerId  int64
Pclass       int64
Sex          object
SibSp        int64
Survived     float64
Ticket       object
dtype: object
```

Fix the data types

In [74]: *# Convert 'Age' and 'Survived' to category Datatype*

```
df['Age'] = df['Age'].astype('int64')
df['Survived'] = df['Survived'].astype('int64')

# Check the new data types
print(df.dtypes)
```

```

Age          int64
Embarked     object
Fare         float64
Name         object
Parch        int64
PassengerId  int64
Pclass       int64
Sex          object
SibSp        int64
Survived     int64
Ticket       object
dtype: object

```

9) Generate descriptive statistics

In [75]: `df.describe()`

Out[75]:

	Age	Fare	Parch	PassengerId	Pclass	SibSp	
count	1309.000000	1309.000000	1309.000000	1309.000000	1309.000000	1309.000000	1
mean	29.685256	33.297261	0.385027	655.000000	2.294882	0.498854	
std	12.899824	51.738919	0.865560	378.020061	0.837836	1.041658	
min	0.000000	0.000000	0.000000	1.000000	1.000000	0.000000	
25%	22.000000	7.895800	0.000000	328.000000	2.000000	0.000000	
50%	29.000000	14.454200	0.000000	655.000000	3.000000	0.000000	
75%	35.000000	31.275000	0.000000	982.000000	3.000000	1.000000	
max	80.000000	512.329200	9.000000	1309.000000	3.000000	8.000000	

10) Visualize and analyze relationships

Univariate analysis: Examine individual features using visualizations.

In [90]: `df.Survived.value_counts(normalize=True)`

```

survival_ratio = df.Survived.value_counts(normalize=True)
print('Percentage of Survivors : ', round(survival_ratio[1]*100, 0), '%')
print('Percentage of Dead      : ', round(survival_ratio[0]*100, 0), '%')

```

```

Percentage of Survivors : 26.0 %
Percentage of Dead      : 74.0 %

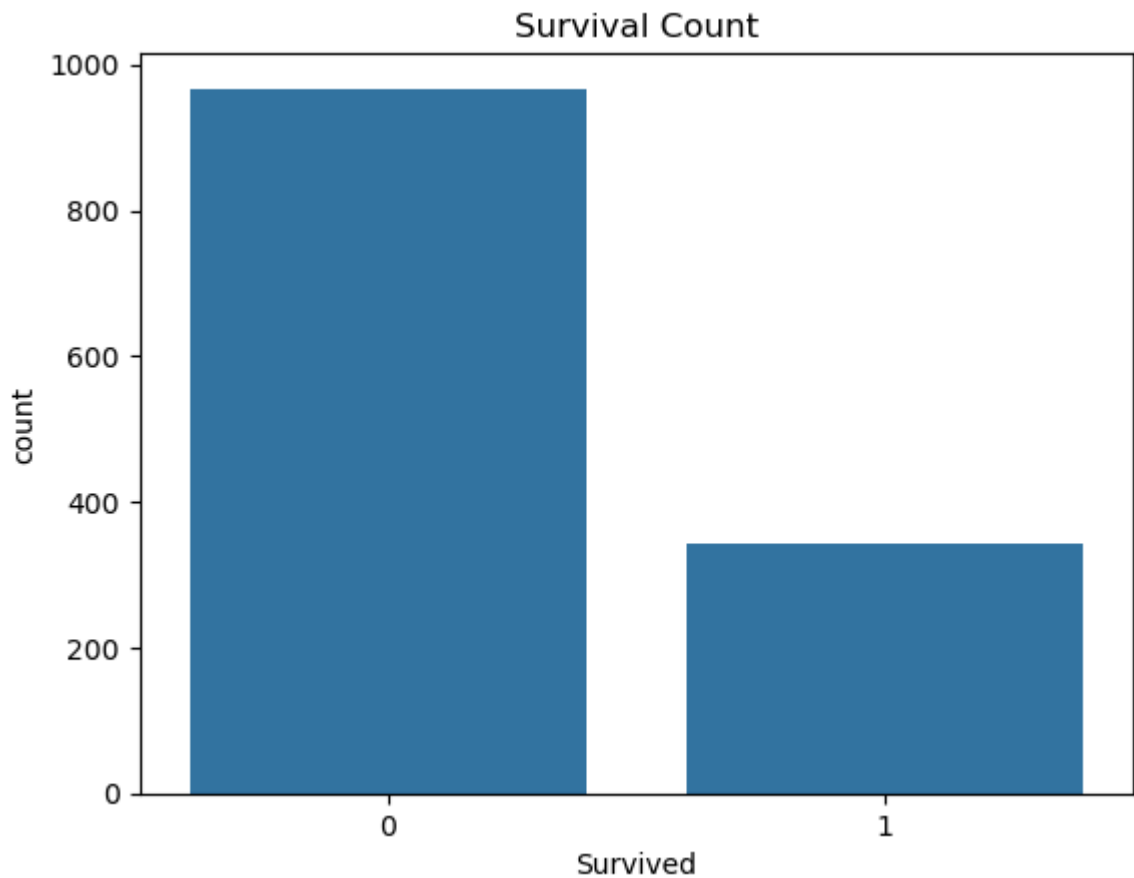
```

In [77]: `# Survival count`

```

sns.countplot(x='Survived', data=df)
plt.title('Survival Count')
plt.show()

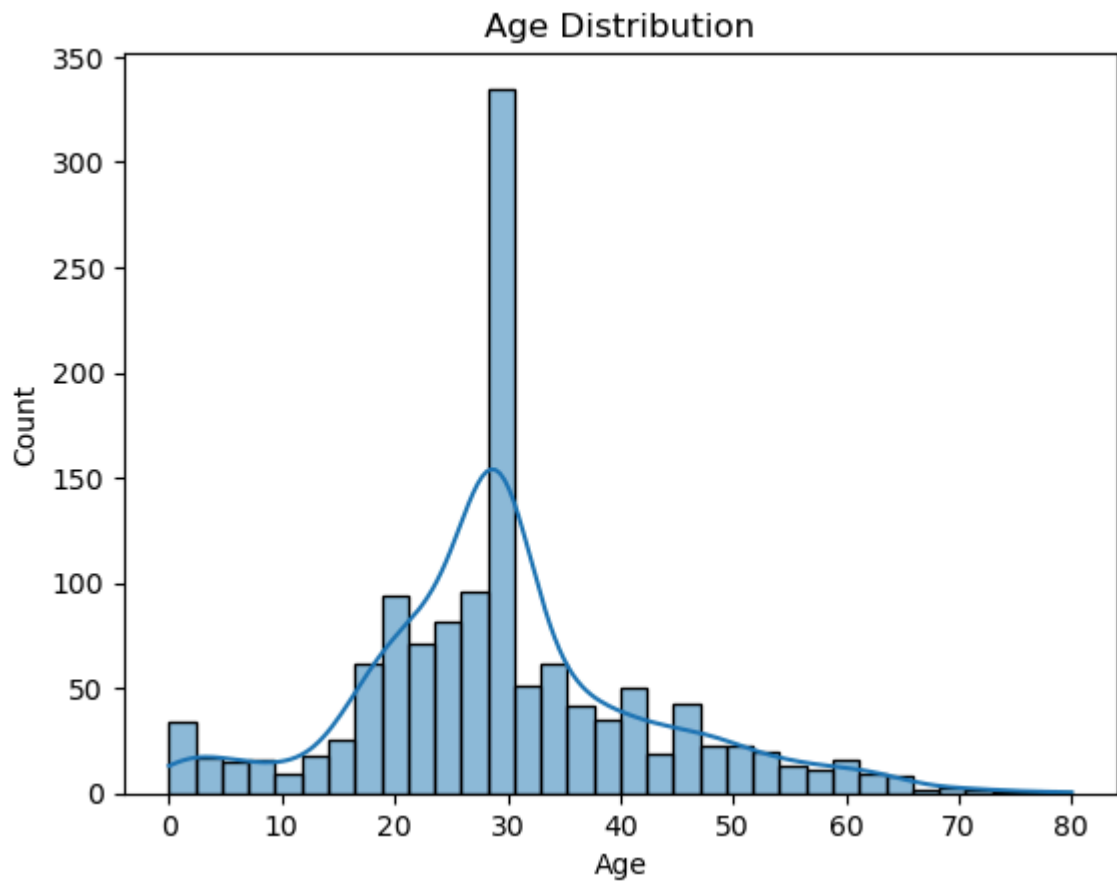
```



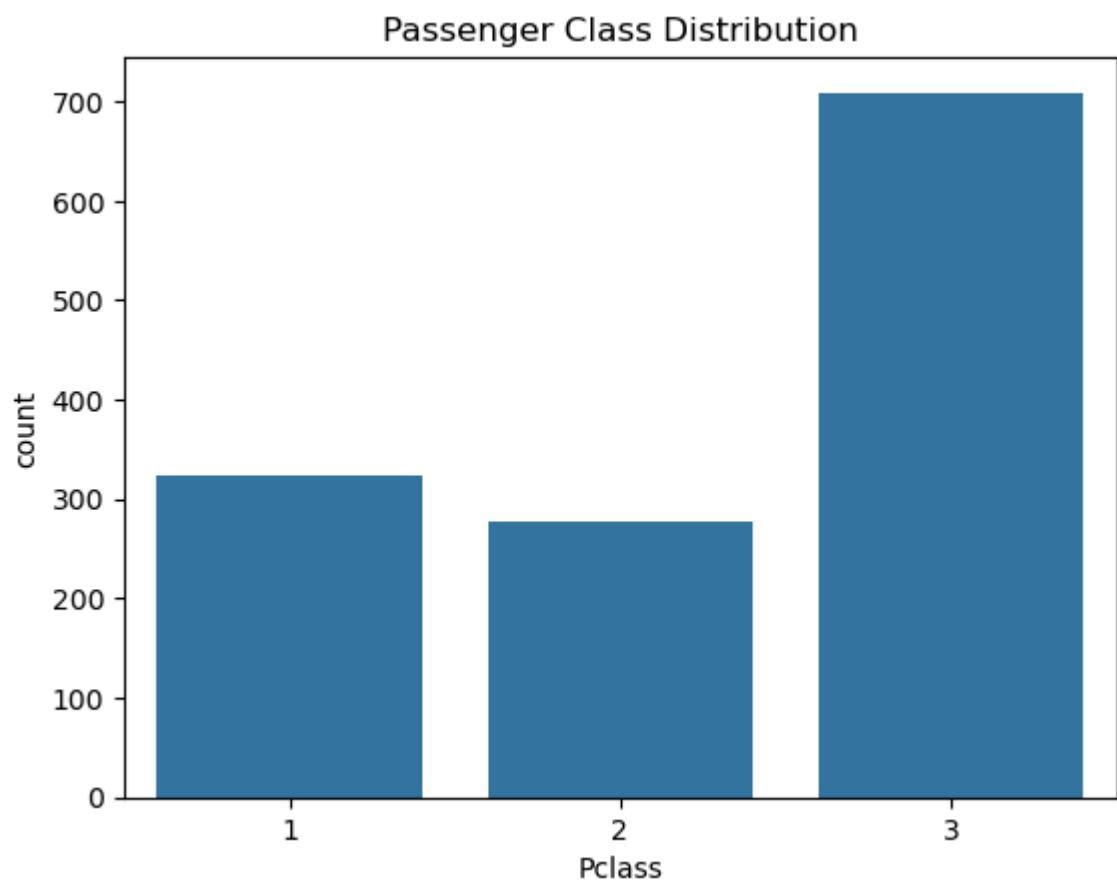
In Survival Count plot, it's evident that the number of non-survivors is greater than survivors.

Only 26% of the passengers were survived, where as a majority 74% of the passenger did not survive the disaster.

```
In [78]: # Age distribution
sns.histplot(df['Age'], kde=True)
plt.title('Age Distribution')
plt.show()
```



```
In [79]: # Passenger Class distribution
sns.countplot(x='Pclass', data=df)
plt.title('Passenger Class Distribution')
plt.show()
```



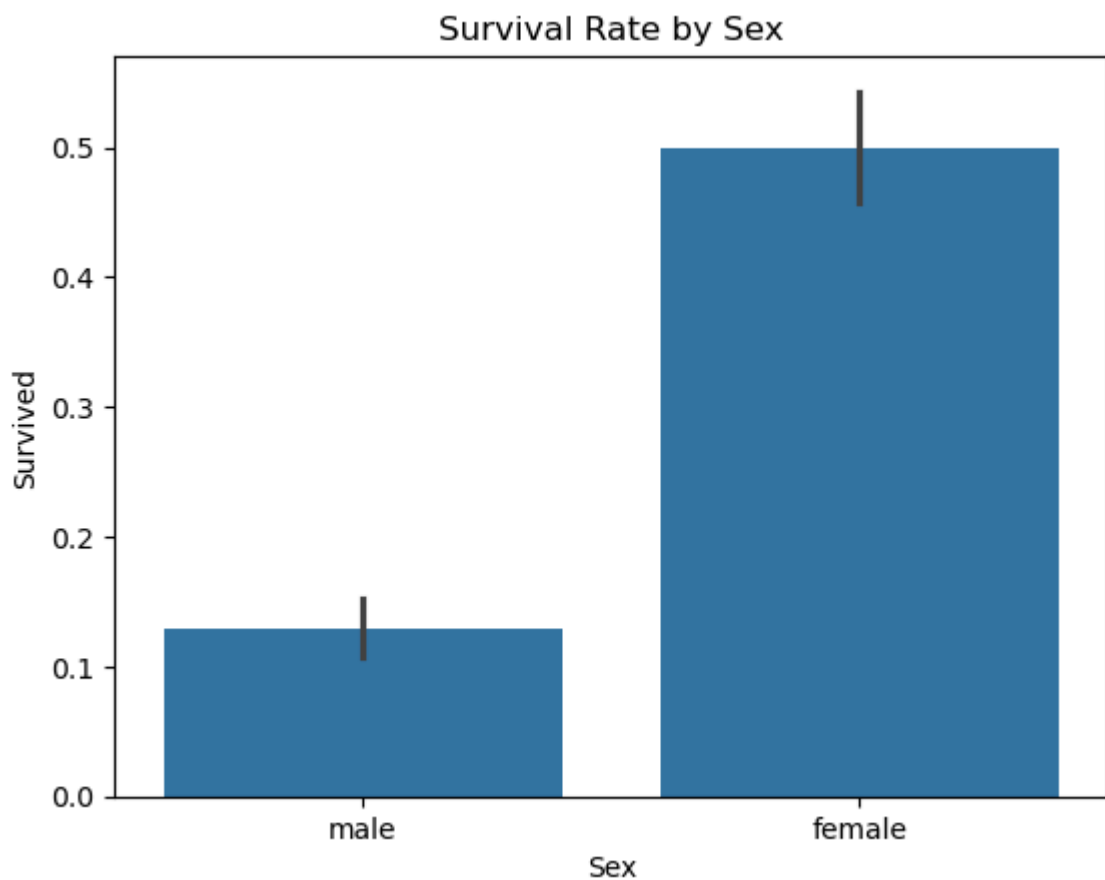
Bivariate analysis: Explore relationships between features and the target variable ('Survived').

```
In [97]: Survival_by_sex = df.groupby('Sex')[['Survived']].mean()

Survival_percentage_by_sex = Survival_by_sex * 100
print(Survival_percentage_by_sex)
```

```
Survived
Sex
female  50.000000
male    12.930012
```

```
In [80]: # Survival by Sex
sns.barplot(x='Sex', y='Survived', data=df)
plt.title('Survival Rate by Sex')
plt.show()
```



We can clearly see that male survival rates is around 13% where as female survival rate is about 50% which suggests that gender has a strong relationship with the survival rates.

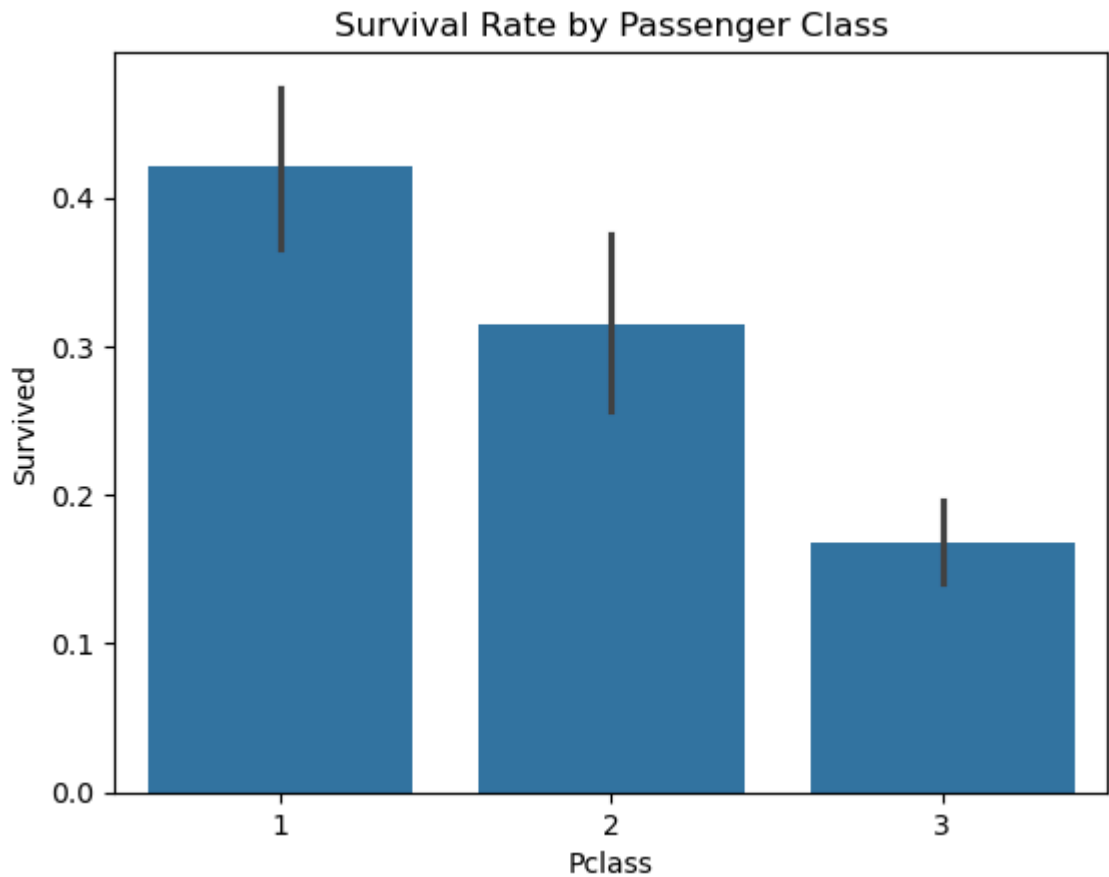
```
In [98]: # Survival by Pclass

Survival_by_Pclass = df.groupby('Pclass')[['Survived']].mean()

Survival_percentage_by_Pclass = Survival_by_Pclass * 100
print(Survival_percentage_by_Pclass)
```

Pclass	Survived
1	42.105263
2	31.407942
3	16.784203

```
In [81]: # Survival by Pclass
sns.barplot(x='Pclass', y='Survived', data=df)
plt.title('Survival Rate by Passenger Class')
plt.show()
```



There is also a clear relationship between Pclass and the survival by referring to above plot. Passengers on Pclass1 had a better survival rate of approx 42% whereas passengers on pclass3 had the worst survival rate of approx 17%

```
In [99]: # Survival by Age

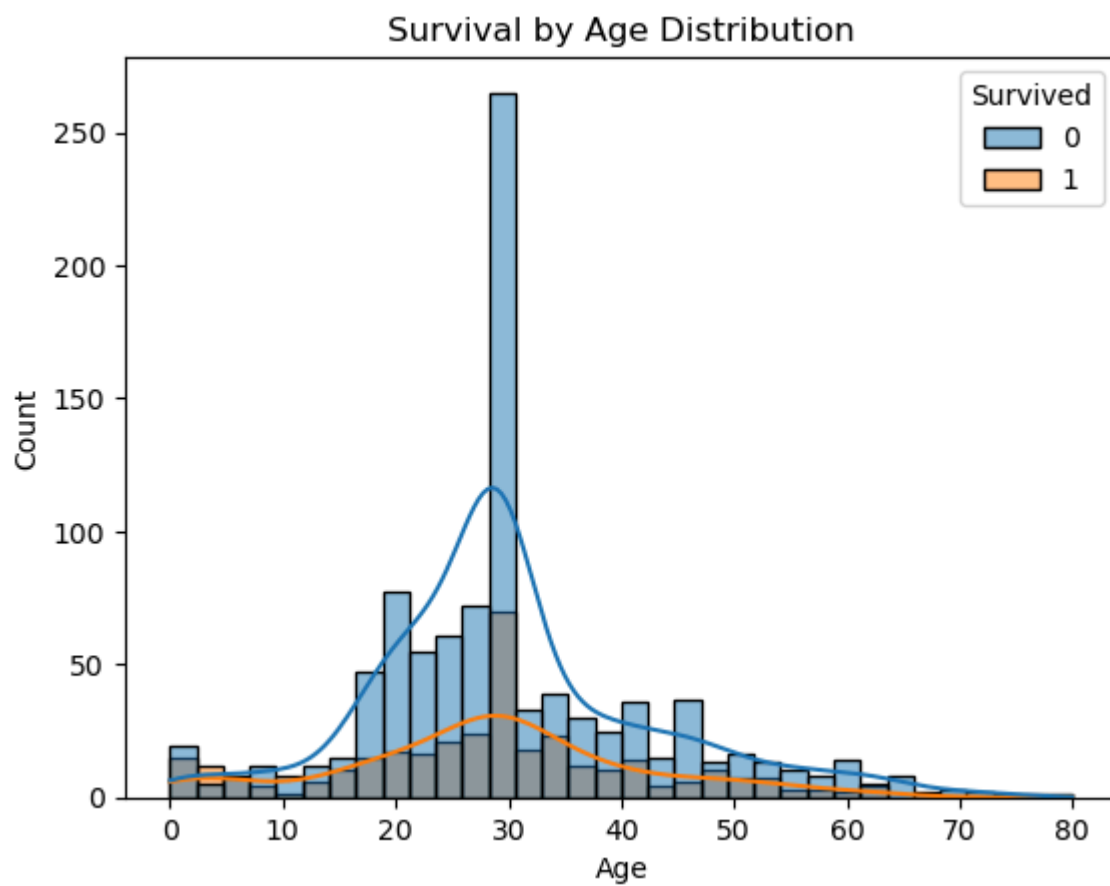
Survival_by_Age = df.groupby('Age')[['Survived']].mean()

Survival_percentage_by_Age = Survival_by_Age * 100
print(Survival_percentage_by_Age)
```


	Survived
Age	
0	58.333333
1	50.000000
2	25.000000
3	71.428571
4	70.000000
..	...
70	0.000000
71	0.000000
74	0.000000
76	0.000000
80	100.000000

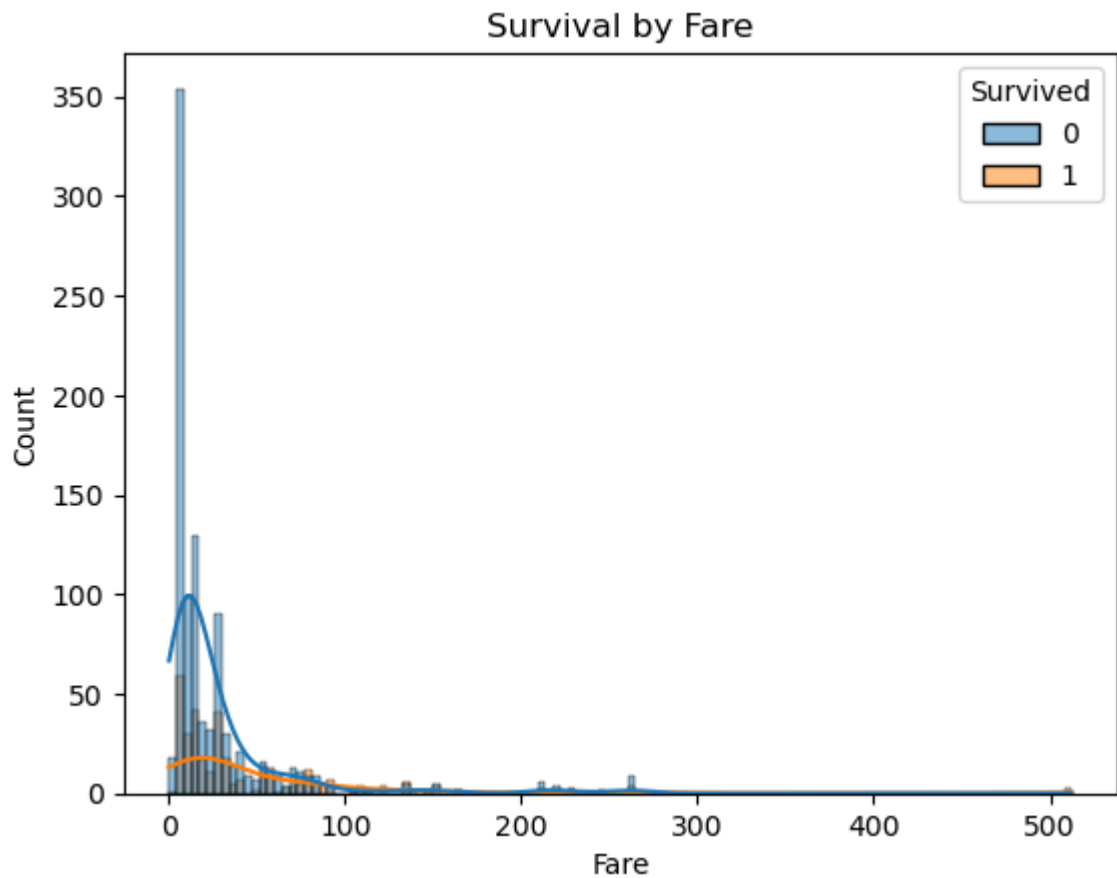
[73 rows x 1 columns]

```
In [82]: # Survival by Age
sns.histplot(x='Age', hue='Survived', data=df, kde=True)
plt.title('Survival by Age Distribution')
plt.show()
```



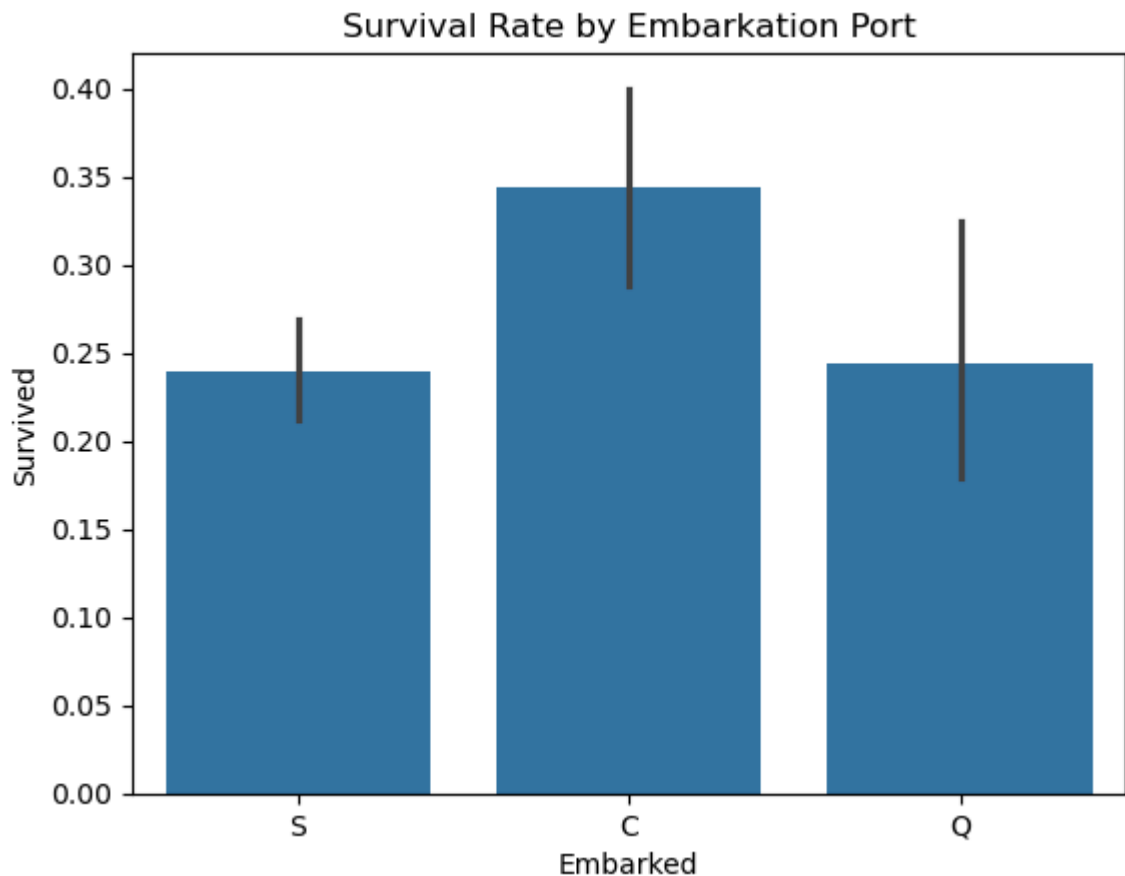
Children and younger individuals might have had a higher chance of survival.

```
In [100... # Survival by Fare
sns.histplot(x='Fare', hue='Survived', data=df, kde=True)
plt.title('Survival by Fare')
plt.show()
```



There is also a marginal relationship between the fare and survival rate.

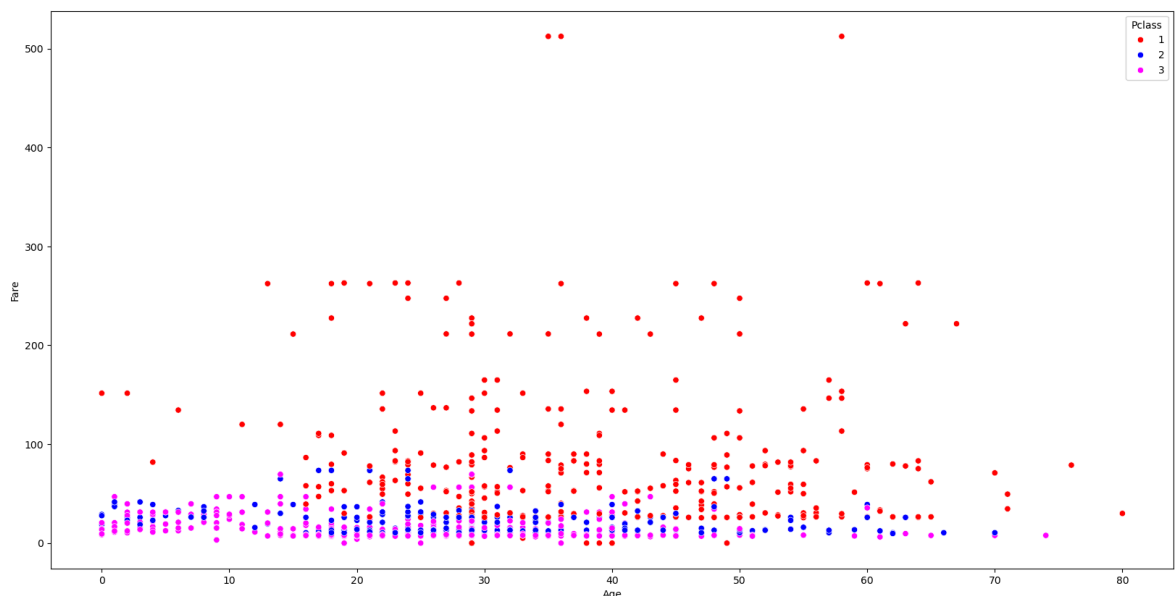
```
In [83]: # Survival by Embarked Port
sns.barplot(x='Embarked', y='Survived', data=df)
plt.title('Survival Rate by Embarkation Port')
plt.show()
```



```
In [103... #sns.pairplot(df[['Age', 'Fare', 'Survived']], hue='Survived')
#plt.title('Pairplot of Age, Fare, and Survival')
#plt.show()

plt.figure(figsize=(20,10))
sns.scatterplot(data=df,hue='Pclass',y='Fare',x='Age',palette=['red','blue','mag
```

Out[103... <Axes: xlabel='Age', ylabel='Fare'>



From the above plot the fare for Pclass=1 is more at all ages

Multivariate analysis: Use heatmaps to identify correlations between numerical features.

```
In [104... plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```



There is a positive coorelation between Fare and Survived and a negative coorelation between Pclass and Surived

There is a negative coorelation between Fare and Pclass, Age and Plclass

In []: